PROPOSTA DE ESPECIFICAÇÃO E ARQUITETURA

CRM de Atendimento e Orçamentos

Funil de atendimento para pequenos negócios que vendem por contato direto

Clean Architecture • DDD • TDD

Laboratório de Programação Web · CEFET-MG

APRESENTAÇÃO

VISÃO GERAL

Roteiro da apresentação

01 **Problema e escopo**

Contexto, funil e requisitos

02 **Modelagem do domínio**

Casos de uso, classes, estados e persistência

03 **Fluxos e arquitetura**

BCE, sequências, atividades e componentes

04 **Implementação**

Decisões, DDD, TDD, backlog e rastreabilidade

SEÇÃO 01

Contexto e escopo

O problema

Atendimentos espalhados em caderno, planilha e conversa geram orçamentos esquecidos, clientes sem retorno e pouca visão do funil.

Público-alvo

Donos de pequenos negócios, atendentes, gestores e administradores

Método

UML, DDD, Clean Architecture e TDD

Unidade central

Orçamento é o aggregate root do funil. Um Cliente pode acumular vários orçamentos e interações ao longo do tempo.

SEÇÃO 01

Funil de atendimento

01 Novo contato

02 Em negociação

03 Orçamento enviado

04 Fechado

PERDIDO Pode ser alcançado a partir de qualquer estágio ativo

Regra central **A validação de transição vive no domínio, não somente na interface.**

SEÇÃO 02

Stack tecnológico

Backend

Python 3.12+
FastAPI + Pydantic
SQLAlchemy 2.x
Alembic

Frontend

Next.js App Router
TypeScript estrito
Tailwind CSS
shadcn/ui + dnd-kit

Dados e testes

PostgreSQL 16
pytest
Vitest + Testing Library
Testes de arquitetura

INFRAESTRUTURA

Docker Compose · variáveis em .env · regra de negócio isolada dos frameworks

SEÇÃO 03

Requisitos funcionais

Entrada e validação

RF01 Registrar contato
RF09 Validar telefone e e-mail
RF08 Permitir vários orçamentos por cliente

Movimentação do funil

RF02 Mover estágio em sequência
RF03 Exigir valor antes de fechar
RF05 Exibir o Kanban

Histórico e métricas

RF04 Registrar interação
RF07 Consultar histórico cronológico
RF06 Calcular volume aberto e taxa de fechamento

Administração

RF10 Gerenciar usuários
Administrador cria, edita e desativa contas, sem desativar a própria conta.

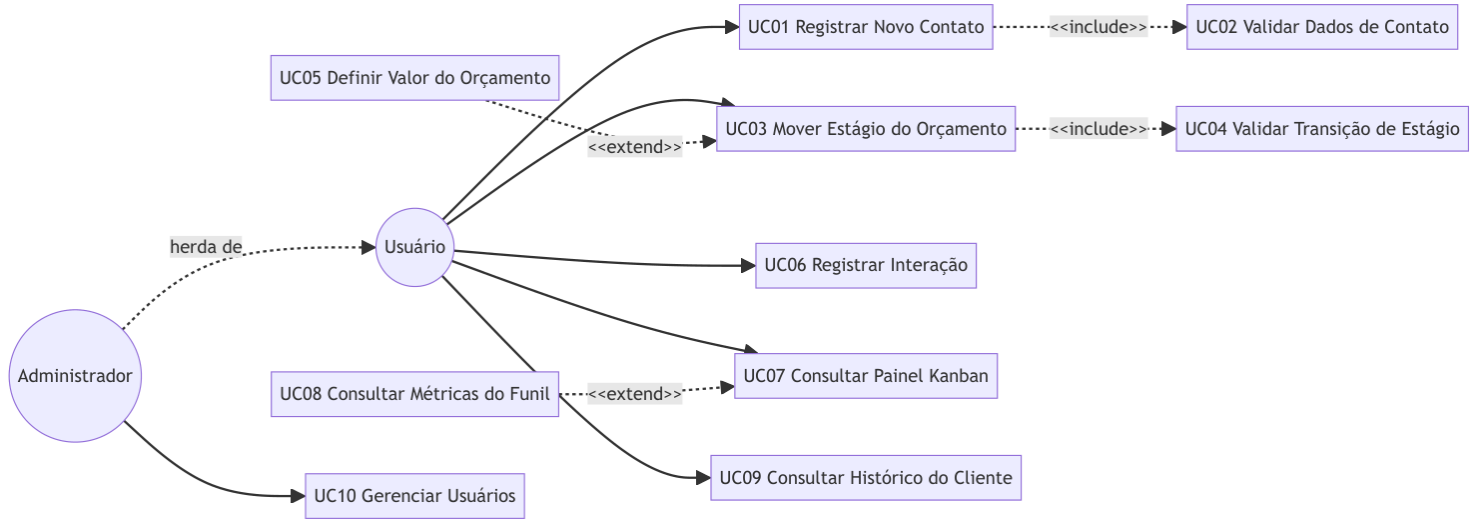
SEÇÃO 03

Requisitos não funcionais que dirigem a arquitetura

RNF01	**Desempenho**	Kanban em menos de 2 s com 500 orçamentos
RNF02	**Usabilidade**	Drag-and-drop com rollback em caso de rejeição
RNF03/06	**Manutenibilidade**	Use Cases dependem de interfaces, não de frameworks
RNF04/08	**Confiabilidade**	Transições e exclusões preservam integridade
RNF05/07/09	**Operação**	Tipagem estrita, autenticação e Docker Compose

SEÇÃO 04

Casos de uso



Include

UC01 sempre valida contato. UC03 sempre valida a transição.

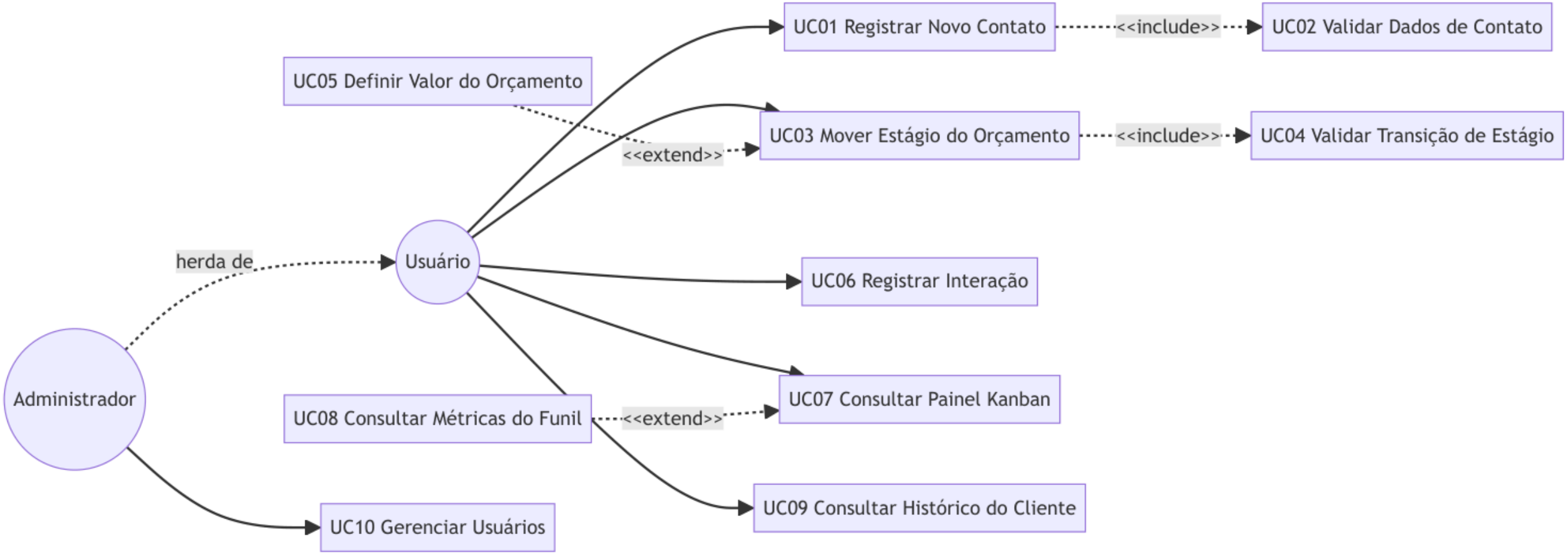
Extend

Definir valor só aparece quando a transição exige valor ainda ausente.

Atores

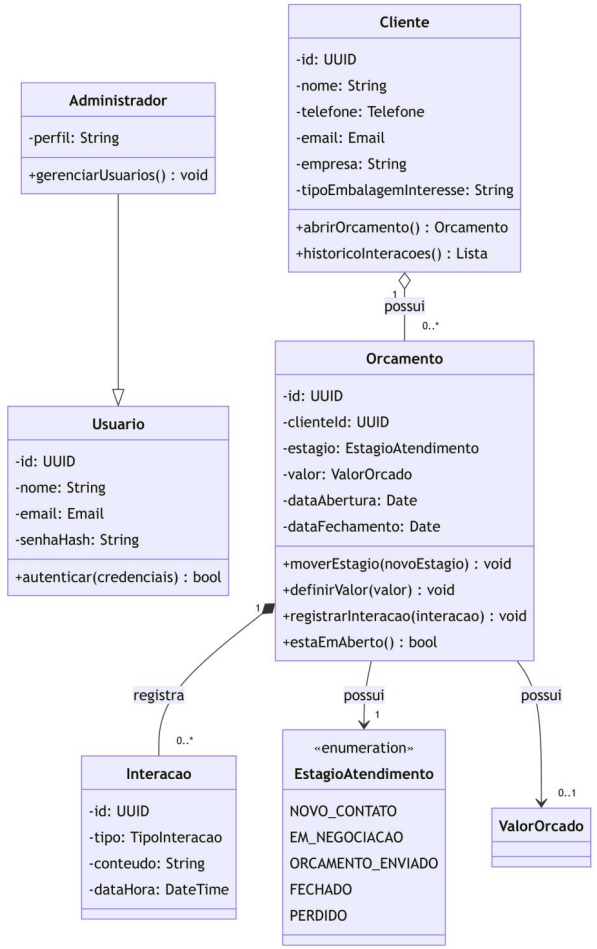
Usuário inicia os fluxos. Administrador herda seus casos de uso e adiciona a gestão de usuários.

Casos de uso


UC01 Registrar Novo Contato
<<include>>
UC02 Validar Dados de Contato
UC05 Definir Valor do Orçamento
<<extend>>
UC03 Mover Estágio do Orçamento
<<include>>
UC04 Validar Transição de Estágio
herda de
Usuário
UC06 Registrar Interação
UC07 Consultar Painel Kanban
Administrador
UC08 Consultar Métricas do Funil
<<extend>>
UC09 Consultar Histórico do Cliente
UC10 Gerenciar Usuários

SEÇÃO 05

Modelo de domínio



Agregados

Cliente e Orçamento são roots separados. Interação pertence ao Orçamento.

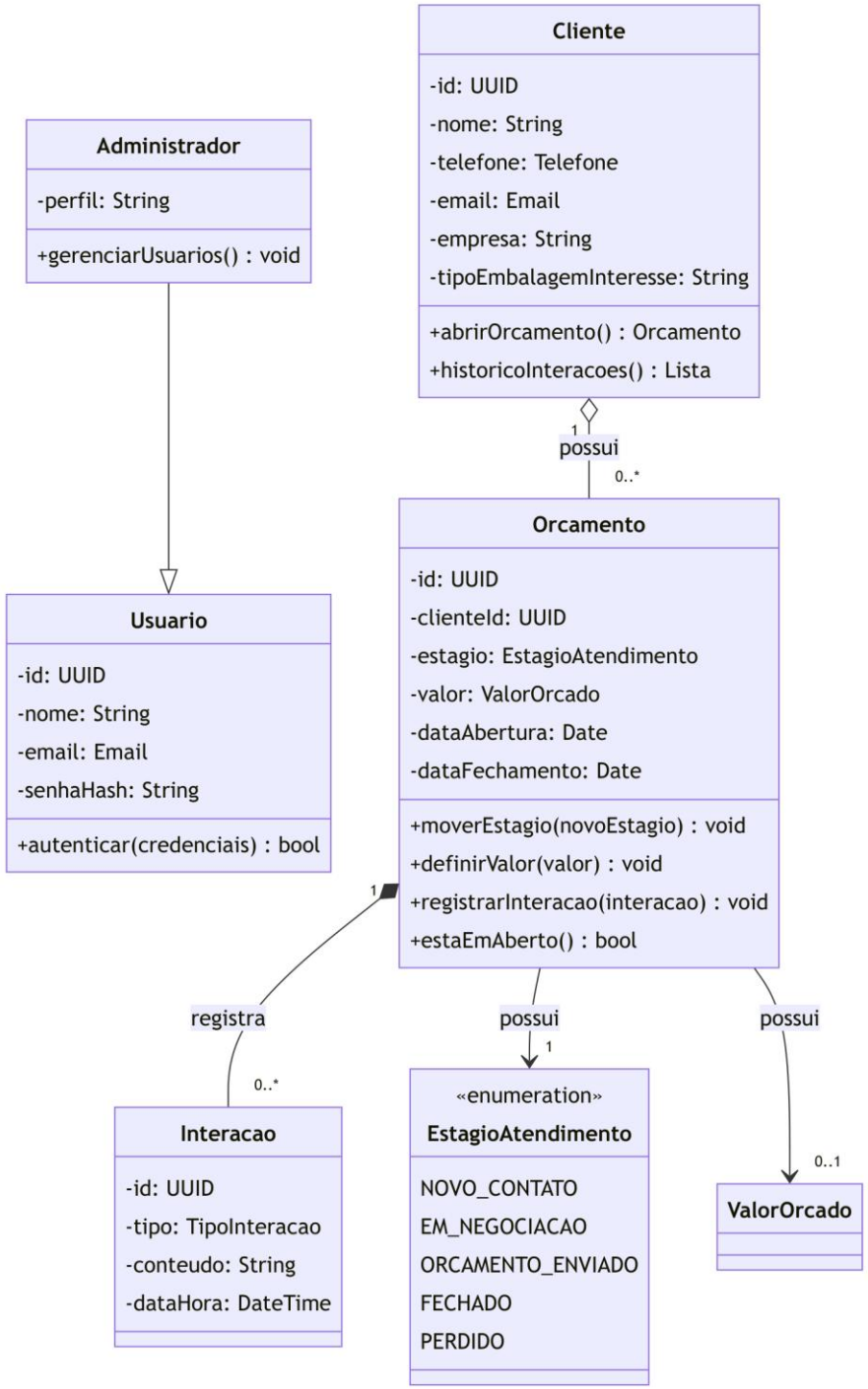
Value Objects

Telefone, E-mail, ValorOrçado e Estágio validam invariantes na construção.

Regra principal

Orçamento.moverEstagio() concentra as transições e protege o funil.

Modelo de domínio



SEÇÃO 05

Persistência

Classe	Persistente?	Estratégia
Cliente	Sim	Tabela cliente · aggregate root
Orçamento	Sim	Tabela orçamento · FK cliente_id
Interação	Sim	Tabela interação · composição e cascade
Usuário / Administrador	Sim	Tabela única · coluna perfil
Estágio	Não	Enum gravado em orçamento
Telefone / E-mail / Valor	Não	Value Objects em colunas embutidas

A regra de transição permanece em Orçamento.moverEstagio(); controller e banco não substituem o domínio.

SEÇÃO 06

Modelo entidade-relacionamento

Usuário → Cliente

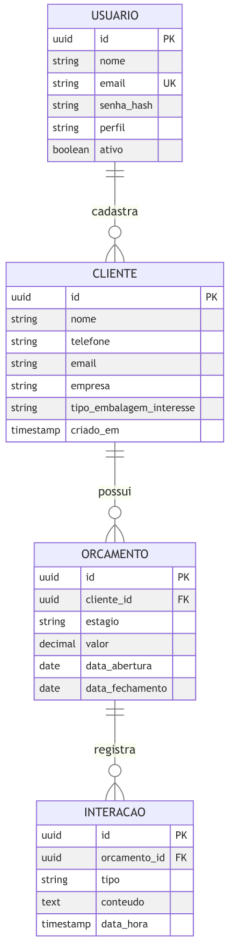
Usuário cadastra clientes. E-mail do usuário é único.

Cliente → Orçamento

Um cliente pode manter vários orçamentos no histórico.

Orçamento → Interação

Interações pertencem ao orçamento e acompanham sua linha do tempo.



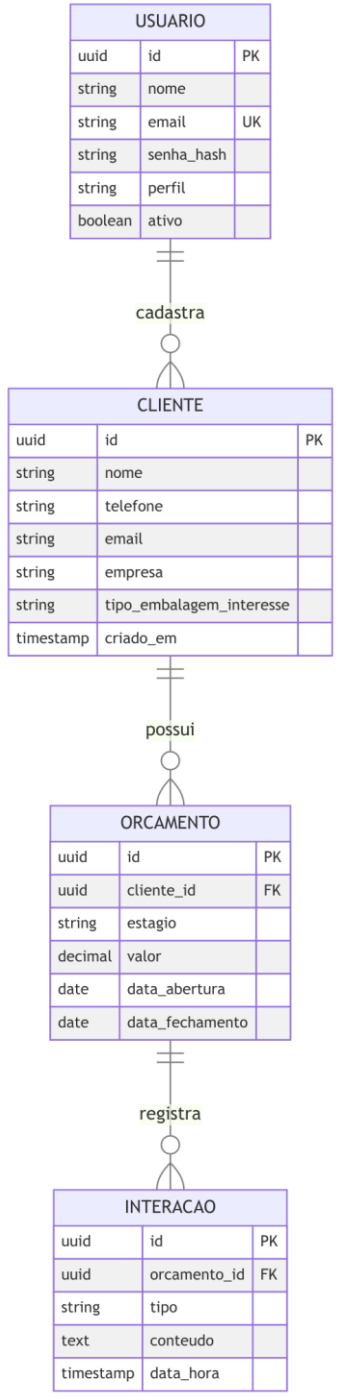
Chaves

UUID como PK
cliente_id e orcamento_id como FKs
E-mail de usuário como UK

Objetos embutidos

Telefone e e-mail viram colunas de Cliente. Valor e estágio viram colunas de Orçamento.

Modelo entidade-relacionamento



SEÇÃO 06

Integridade e índices

Regra	Implementação	Origem
Contato obrigatório	telefone/e-mail NOT NULL + validação	RF09
Estágio válido	NOT NULL + valores do enum	RF02
Valor opcional	NULL é válido até o fechamento	RF03
Excluir cliente	ON DELETE RESTRICT	RF08
Excluir orçamento	Interações em CASCADE	Composição

Índices

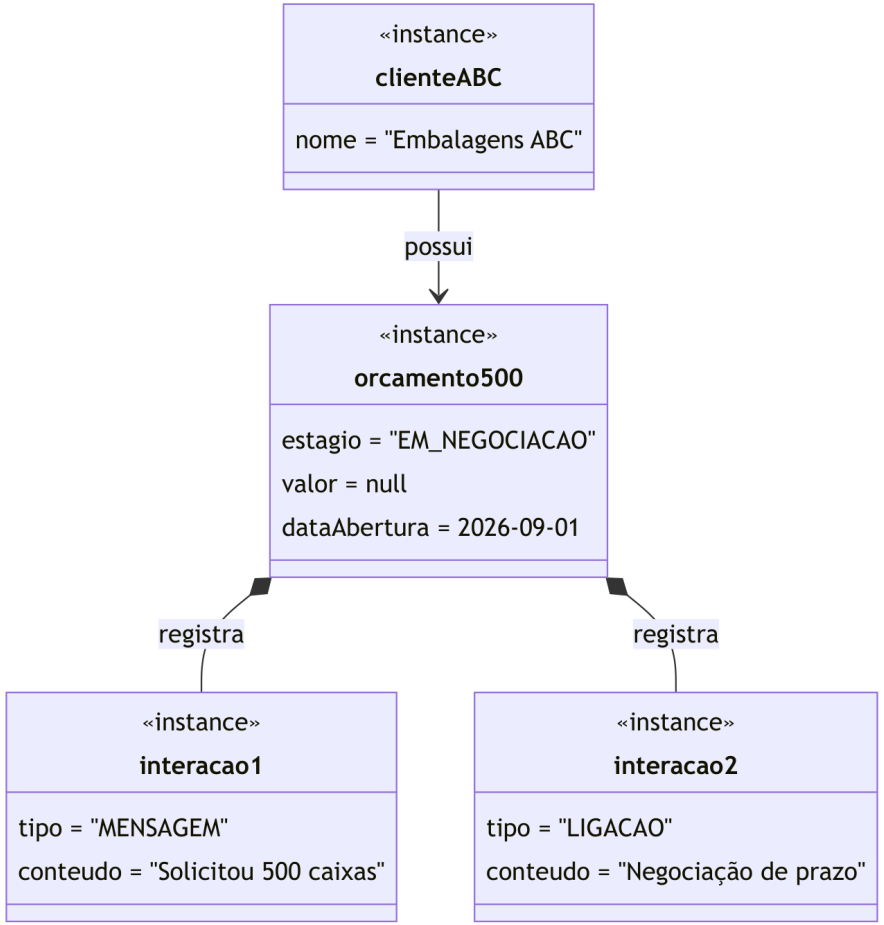
idx_orcamento_estagio
idx_orcamento_cliente_id
idx_interacao_orcamento_data

Por quê

Sustentam o Kanban em menos de 2 s e o histórico em ordem cronológica.

A regra "não fechar sem valor" vive no domínio. O banco atua como última linha de defesa.

Diagrama de objetos: cenário concreto



Embalagens ABC

1. **Orçamento em negociação**

   500 caixas e valor ainda indefinido

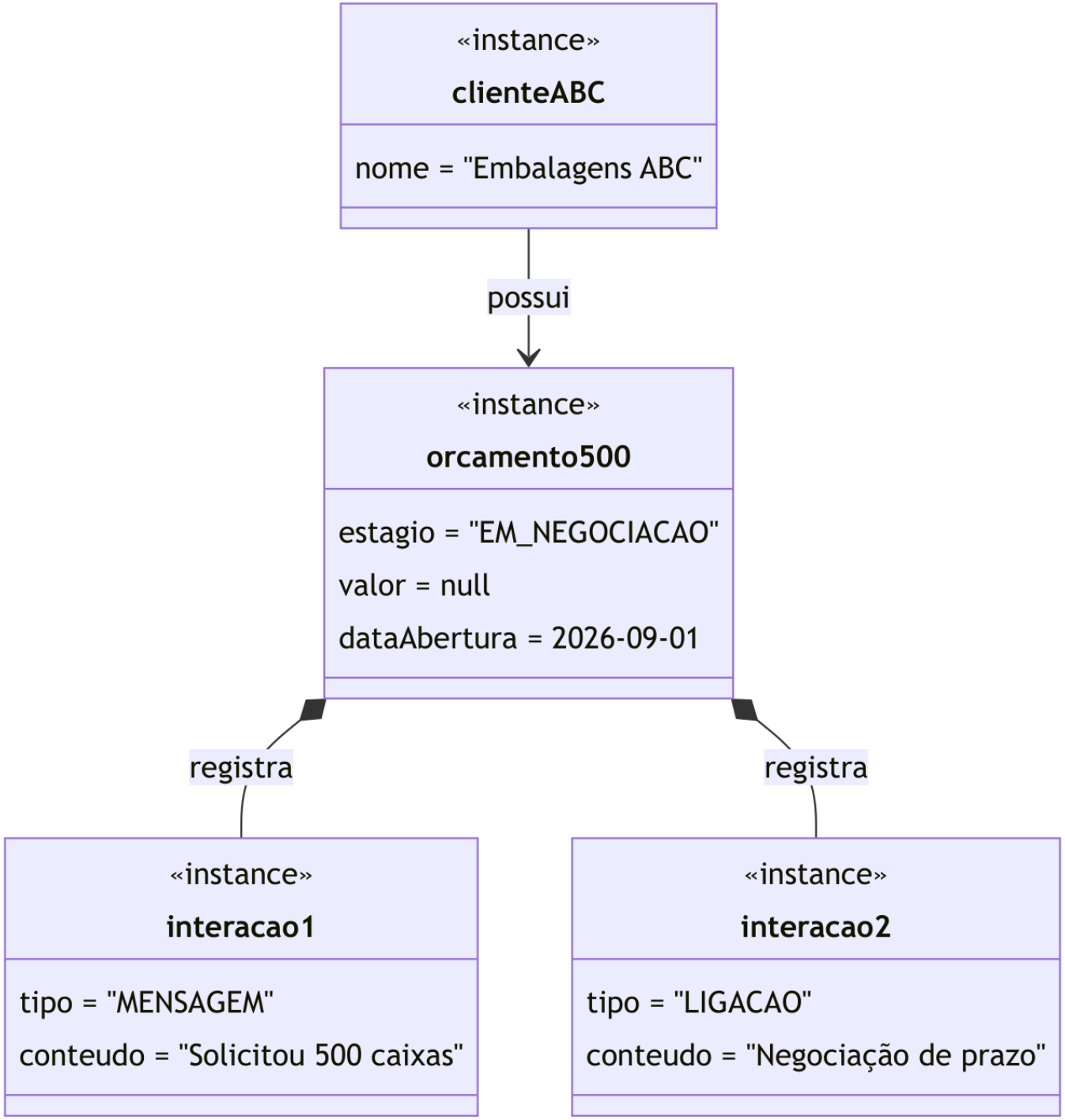
2. **Duas interações**

   Mensagem inicial e ligação sobre prazo

3. **Estado válido**

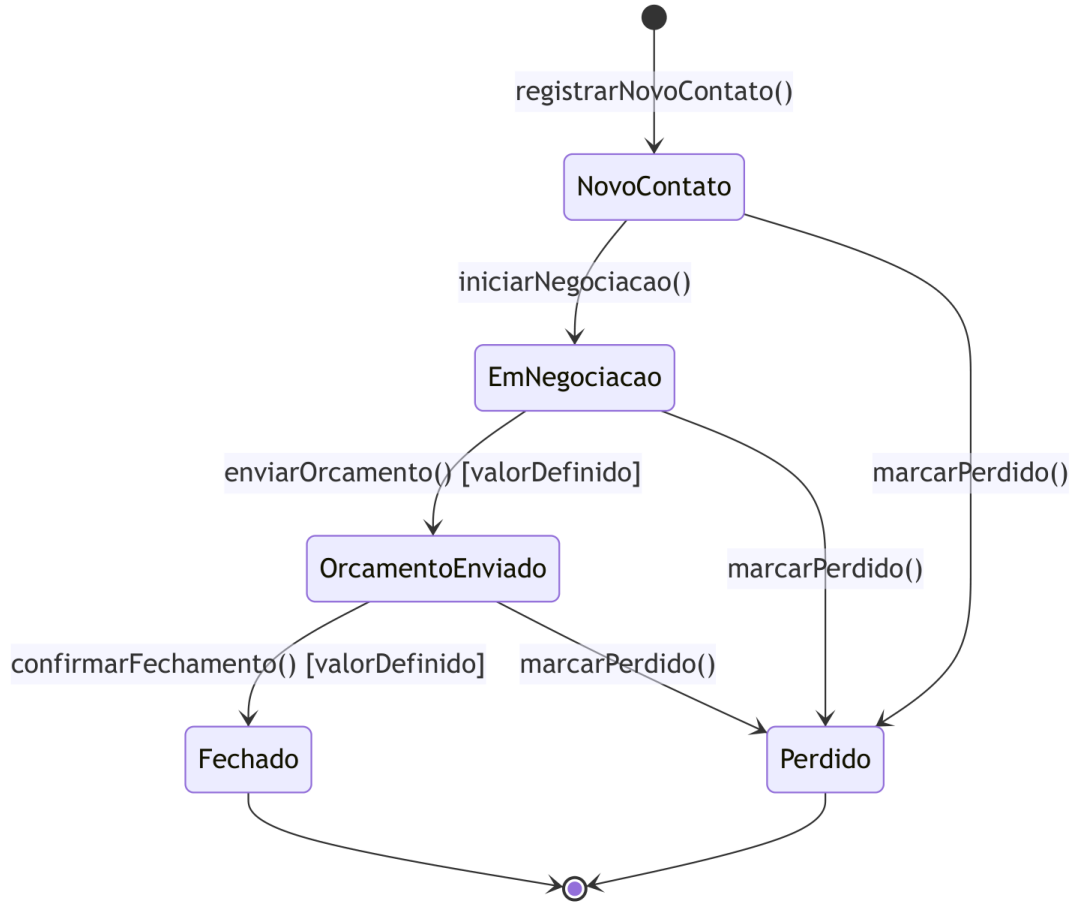
   A guarda de valor bloqueia o avanço para envio ou fechamento

Diagrama de objetos



SEÇÃO 07

Máquina de estados do Orçamento



Sem saltos

Novo Contato não avança direto para Fechado.

Guarda de valor

Envio e fechamento exigem valor definido.

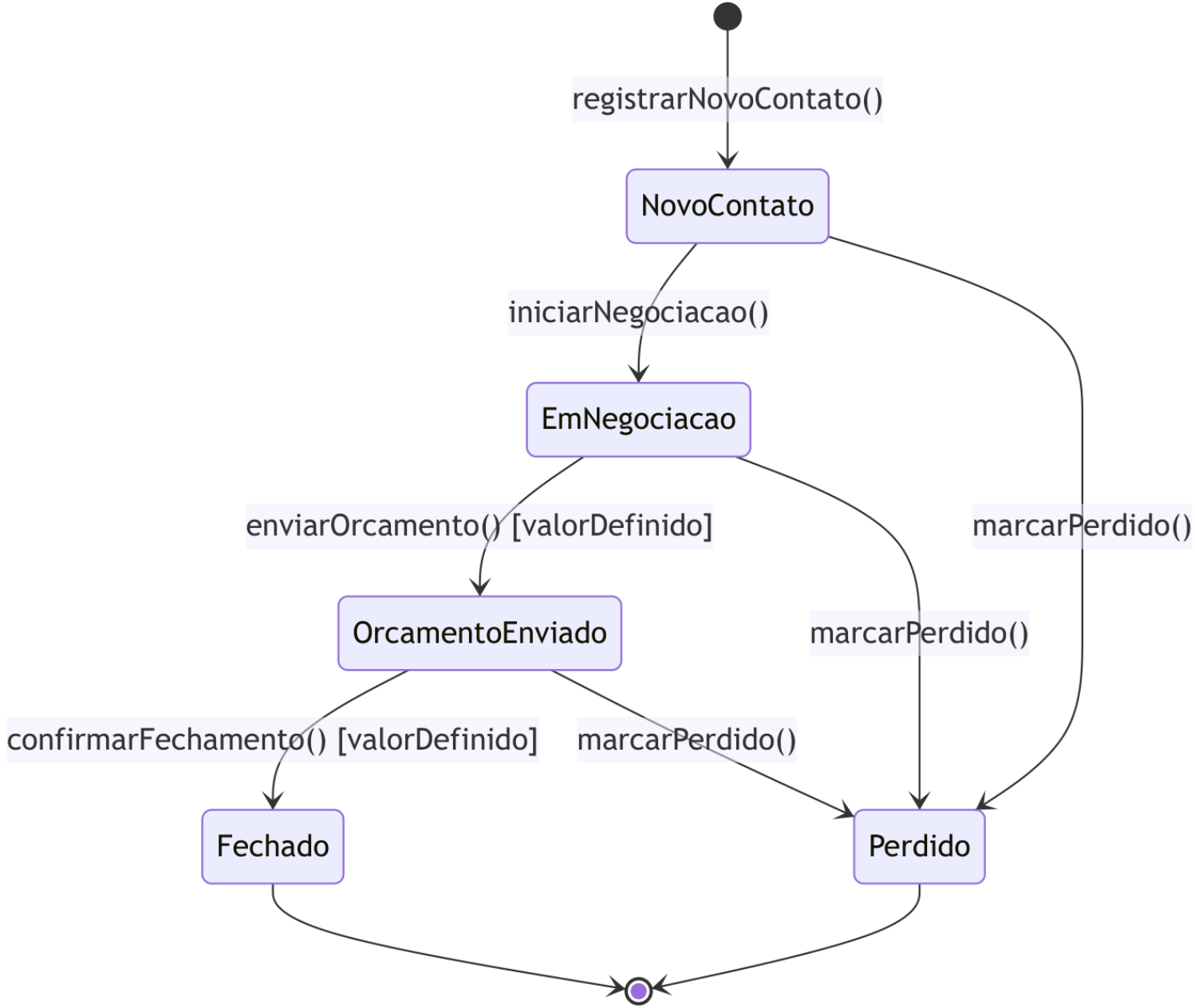
Estados terminais

Fechado e Perdido não reabrem.

Fonte de verdade

Qualquer outro par gera TransicaoInvalida.

Máquina de estados



SEÇÃO 08

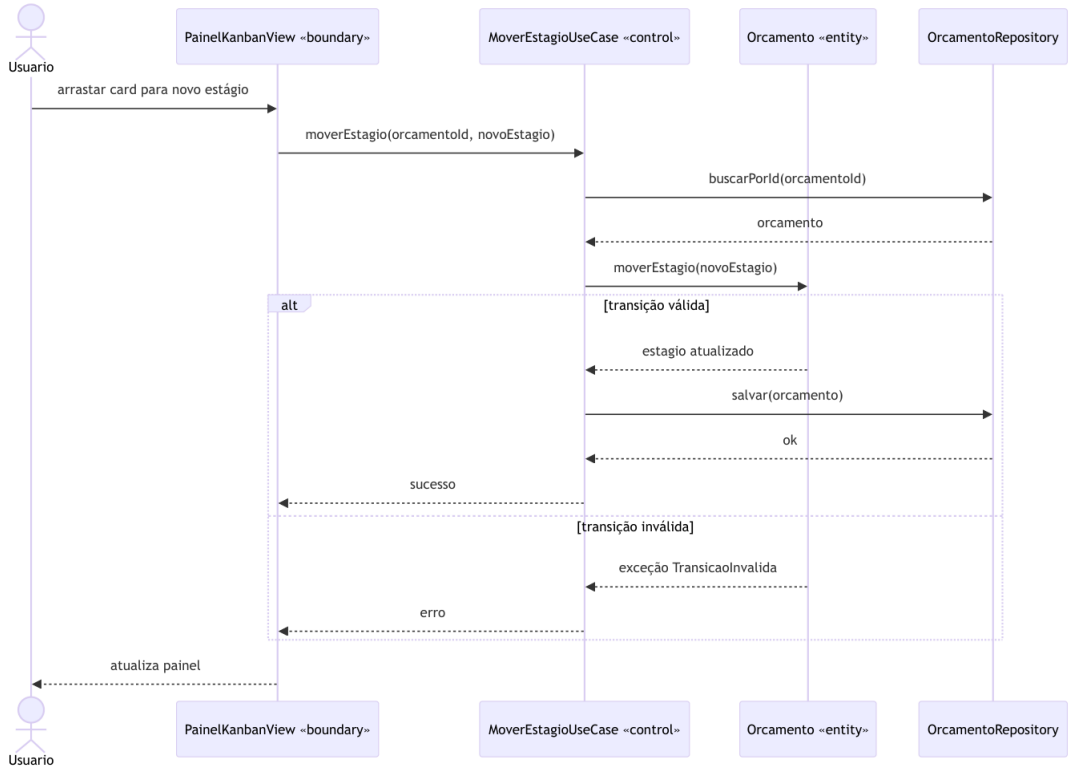
Boundary-Control-Entity

USUÁRIO	BOUNDARY	CONTROL	ENTITY
Registrar contato	ClienteFormView	RegistrarNovoContato	Cliente · Orçamento
Mover estágio	PainelKanbanView	MoverEstagio	Orçamento · Estágio
Registrar interação	InteracaoFormView	RegistrarInteracao	Orçamento · Interação
Consultar métricas	MetricasHeaderView	ConsultarMetricas	Orçamento

Boundary fala com o exterior · Control coordena · Entity protege a regra de negócio

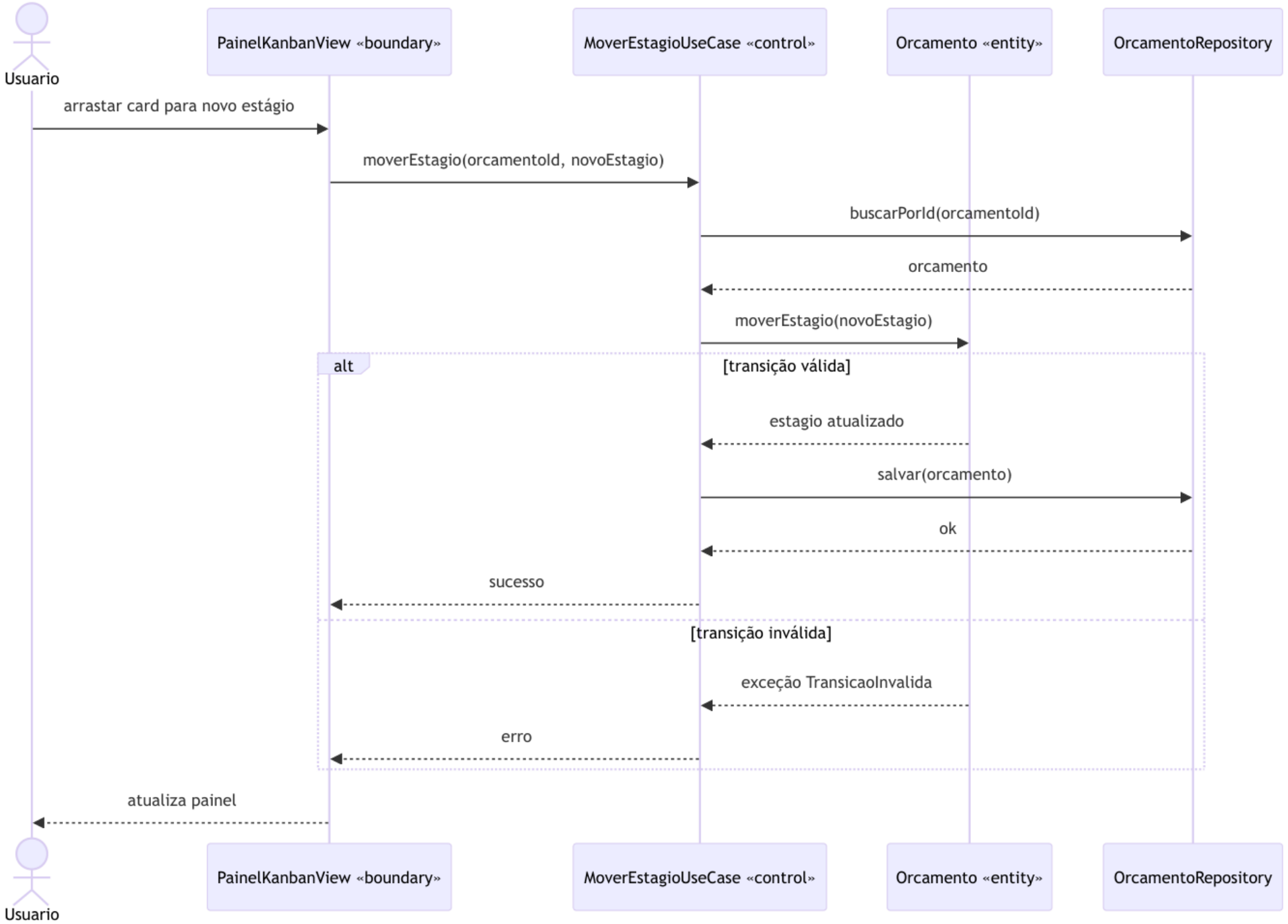
SEÇÃO 09

Sequência crítica: mover estágio



O Use Case carrega o agregado; a entidade decide; o repositório salva somente quando a transição é válida.

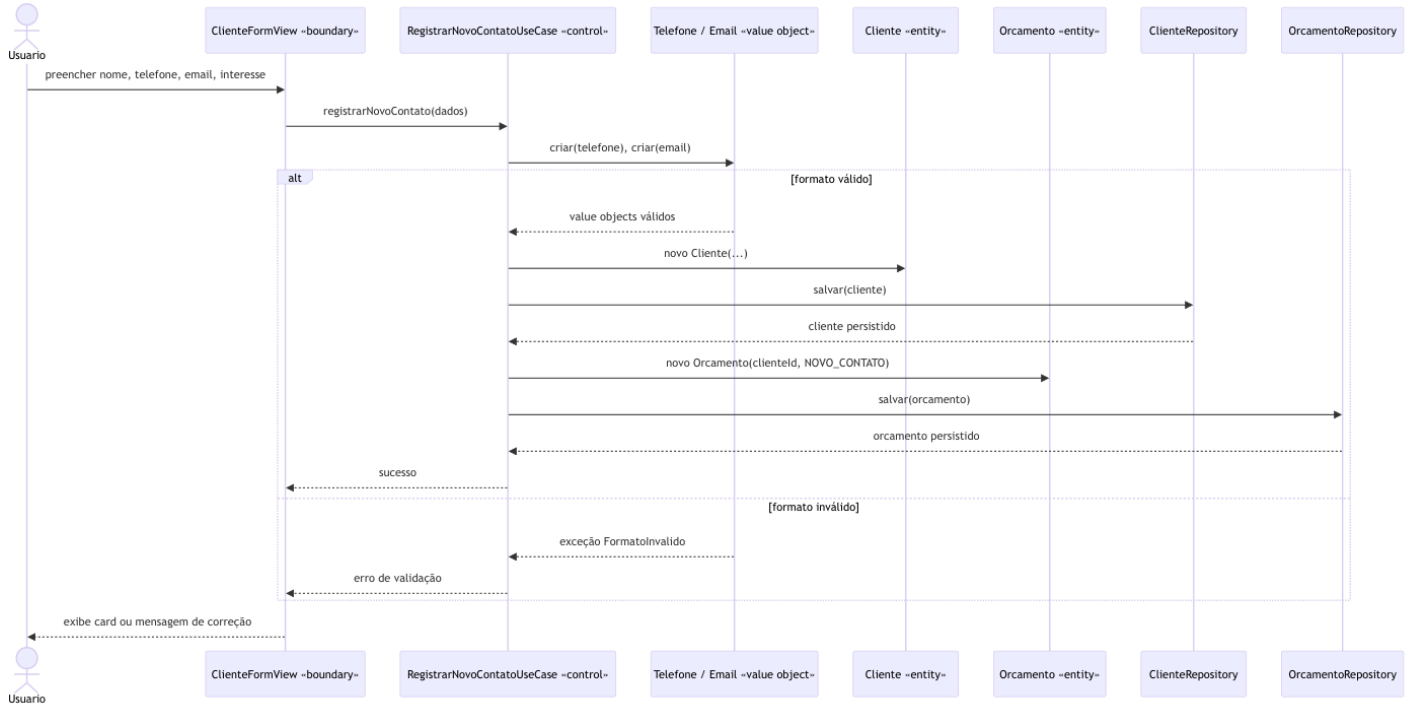
Sequência: mover estágio



SEÇÃO 09

Sequências: registrar contato e interação

UC01 · Registrar novo contato



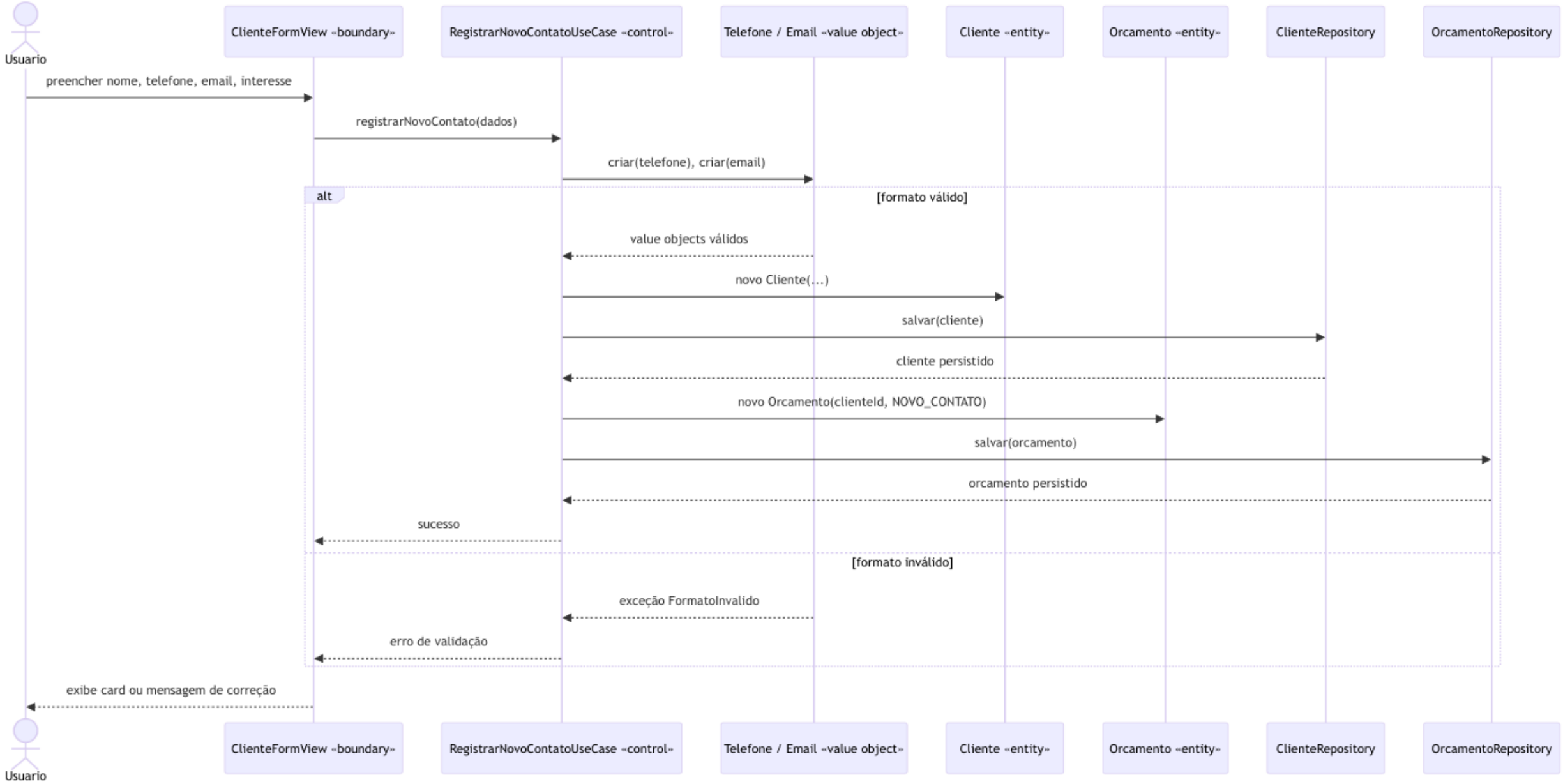
Validação antes da escrita

Telefone e E-mail são construídos antes de persistir. Formato inválido impede a criação de Cliente e Orçamento.

UC06 · Registrar interação

A Interação nasce e é anexada pelo Orçamento. Não existe InteracaoRepository, preservando a composição do agregado.

Sequência: registrar contato



SEÇÃO 10

Fluxo ponta a ponta

1

Registrar contato

Validar telefone e e-mail

2

Novo contato

Criar orçamento

3

Negociar

Registrar interações e definir valor

4

Enviar orçamento

Aguardar decisão do cliente

FECHADO

Cliente aceita

PERDIDO

Cliente recusa ou fluxo é encerrado

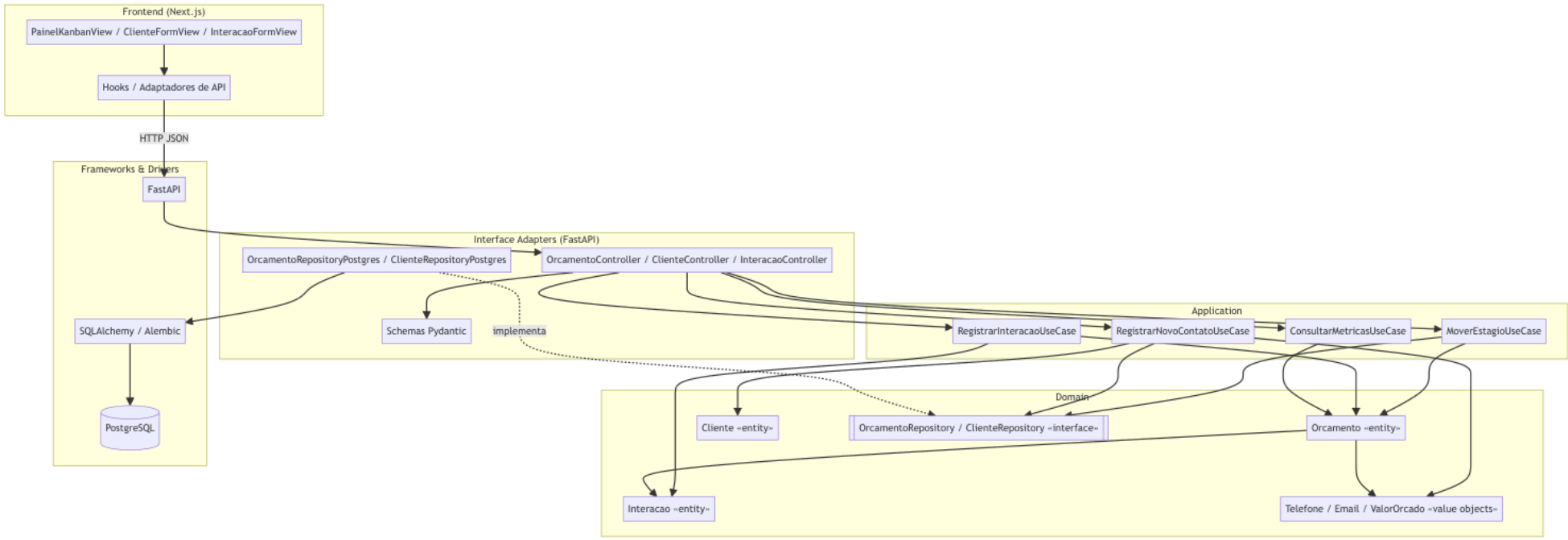
SEÇÃO 10

Movimentação de card por responsabilidade

Usuário	Frontend	Use Case	Orçamento
1 · Arrasta o card	2 · Atualização otimista	4 · Carrega o agregado	5 · Valida transição
8 · Lê a mensagem de erro	3 · Chamada HTTP	6 · Persiste se aceito	5b · Verifica guarda de valor
	7 · Rollback visual		

SEÇÃO 11

Componentes e direção das dependências



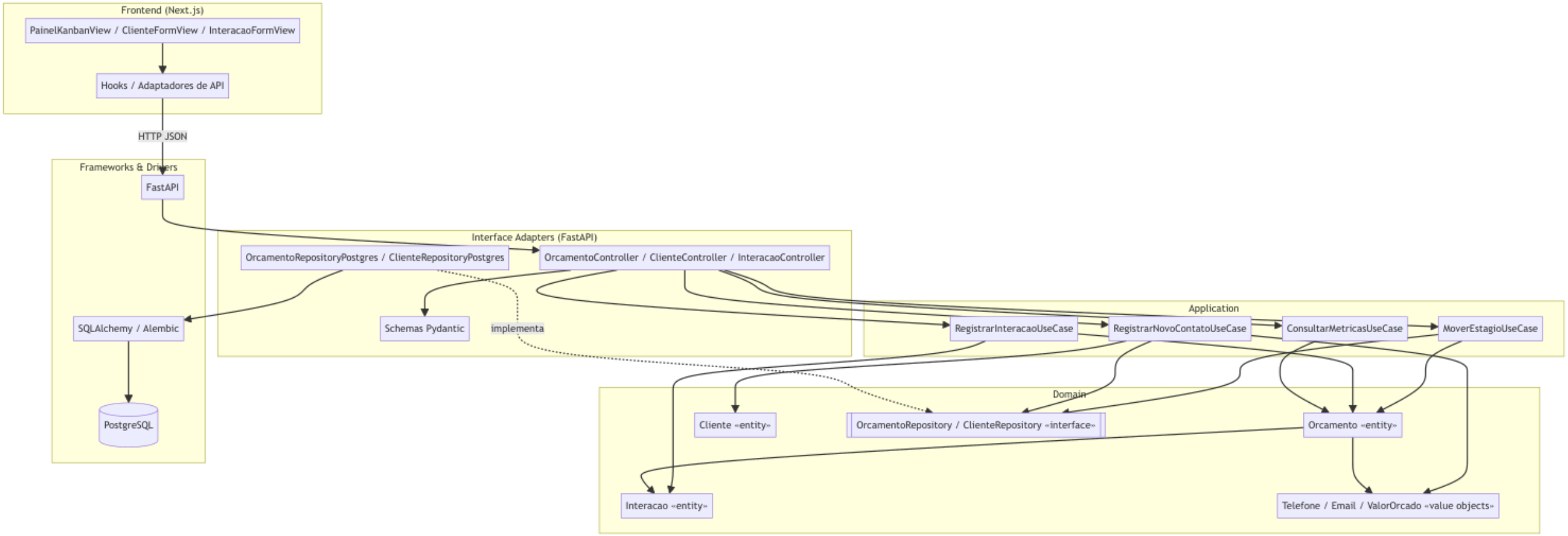
RNF03

Domínio e aplicação não importam FastAPI, Pydantic ou SQLAlchemy.

RNF06

Use Cases dependem de interfaces e podem rodar com repositórios fake.

Diagrama de componentes



SEÇÃO 12

Decisões de arquitetura

Regra no domínio

Uma chamada direta à API também respeita as transições.

Orçamento como root

Interação só existe através do Orçamento.

Agregados separados

Mover um card não exige carregar todo o histórico do cliente.

Estágio como enum

Conjunto fixo evita join em toda consulta do Kanban.

Quatro camadas

domain, application, adapters e infra.

Tipagem estrita

Mudança de contrato falha na compilação.

SEÇÃO 12

Trade-offs e riscos

Trade-offs aceitos

Regra no domínio

Ganha testabilidade; escrita direta no banco poderia contornar a regra.

Enum em código

Adicionar estágio exige deploy e migration; o funil é intencionalmente fixo.

Duas escritas em UC01

Cliente e Orçamento são separados; a transação do Use Case mitiga o risco.

Riscos e mitigação

Regra duplicada	API expõe o mapa de transições
Kanban lento	Índice por estágio e projeção enxuta
Framework no domínio	Teste de arquitetura falha a build
UI otimista divergente	Rollback explícito

SEÇÃO 13

DDD e estrutura de camadas

Aggregate root	Entidade interna	Value Objects	Repository
Cliente	—	Telefone, E-mail	ClienteRepository
Orçamento	Interação	Estágio, Valor	OrcamentoRepository
Usuário	—	E-mail	UsuarioRepository

backend/src/

domain/	entities · value_objects · exceptions · repository interfaces
application/	use_cases
adapters/	controllers · repositories Postgres · schemas Pydantic
infra/	db · config · security · composition root

SEÇÃO 13

Plano de testes TDD

RED → GREEN → REFACTOR



Arquitetura

A build falha se domain/ ou application/ importarem FastAPI, Pydantic ou SQLAlchemy.

SEÇÃO 14

Backlog priorizado por fase

0

Fundação

Camadas, CI e Docker

1

MVP

Funil ponta a ponta

2

Histórico

Interações e índices

3

Medição

Métricas, JWT e admin

4

Evolução

Filtros, busca e exportação

Definição de pronto da Fase 1

Registrar um contato e conduzi-lo até Fechado ou Perdido, com transições inválidas bloqueadas pelo domínio.

Domínio primeiro. Interface por último.

SEÇÃO 15

Checklist e rastreabilidade

Cobertura da proposta

- ✓ **Requisitos e casos de uso**
- ✓ **Classes, objetos e estados**
- ✓ **Persistência e DER**
- ✓ **BCE e sequências**
- ✓ **Atividades e componentes**
- ✓ **DDD, TDD e backlog**

Amostra de rastreabilidade

Req.	Caso de uso	Teste
RF02	UC03	Rejeita salto de etapa
RF03	UC03/05	Exige valor
RF09	UC02	Recusa formato inválido
RNF01	UC07	Carga com 500 ativos
RNF03	—	Import proibido falha a build

Resultado: uma proposta rastreável, organizada por domínio e pronta para orientar o MVP.